BIZEV-05: KPIs and Metrics

> **Series:** BIZEV — Business Events & Funnel Analysis | **Notebook:** 5 of 7 | **Created:** March 2026 | **Last Updated:** 10/25/2026

Overview

Business events contain the raw signals needed to compute meaningful Key Performance Indicators (KPIs) — transaction volume, average order value, error rates by business process, and more. This notebook shows how to define and compute business KPIs from event data using DQL, extract metrics from business events via OpenPipeline for long-term retention, build composite business health scores, and perform trend analysis to detect shifts in business performance.

Table of Contents

1. Defining Business KPIs
2. Transaction Volume KPIs
3. Revenue KPIs
4. Error Rate KPIs
5. Metric Extraction via OpenPipeline
6. Business Health Score
7. Trend Analysis
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:bizevents:read`, `storage:metrics:read`
Data	Business events with transaction types and value fields
Knowledge	BIZEV-01 through BIZEV-03, DQL `summarize` and `makeTimeseries`

1. Defining Business KPIs

A KPI is a quantifiable measure of business performance. The best KPIs are:

Property	Description
Specific	Tied to a concrete business outcome
Measurable	Calculable from available data
Actionable	Teams can influence the metric
Time-bound	Measured over a defined period

Common Business Event KPIs

KPI	Formula	Business Question
Transaction Volume	`count()` of order events	How many orders per hour/day?
Average Order Value (AOV)	`avg(amount)` of order events	How much per order on average?
Conversion Rate	Orders / Views * 100	What % of visitors buy?
Error Rate	Failed / Total * 100	What % of transactions fail?
Revenue per Hour	`sum(amount)` per hour	What is the hourly revenue run rate?
Time to Complete	Avg time from start to end event	How long does a transaction take?

2. Transaction Volume KPIs

Transaction volume is the most fundamental business KPI — it tells you how active your business is.

```
// Transaction volume by event type over the last 24 hours
fetch bizevents, from:-24h
| summarize volume = count(), by:{event.type}
| sort volume desc
| limit 15
```

```
// Hourly transaction volume trend — spot peak and off-peak hours
fetch bizevents, from:-24h
| makeTimeseries tx_volume = count(), interval:1h
```

```
// Daily transaction volume for the past 7 days by event type
fetch bizevents, from:-7d
| fieldsAdd day = bin(timestamp, 24h)
| summarize daily_volume = count(), by:{day, event.type}
| sort day asc, daily_volume desc
```

3. Revenue KPIs

Revenue KPIs require an `amount` or `total` field in your business events. These are the metrics executives care about most.

> **Note:** Replace `amount` with the actual field name in your business events that carries the monetary value.

```
// Revenue KPI summary for the last 24 hours
fetch bizevents, from:-24h
| filter event.type == "com.myapp.order.completed"
| filter isNotNull(amount)
| summarize {total_revenue = sum(toDouble(amount)),
           avg_order_value = avg(toDouble(amount)),
           median_order_value = median(toDouble(amount)),
           max_order_value = max(toDouble(amount)),
           order_count = count()}
```

```
// Revenue timeseries — hourly revenue over the last 24 hours
fetch bizevents, from:-24h
| filter event.type == "com.myapp.order.completed"
| filter isNotNull(amount)
| makeTimeseries hourly_revenue = sum(toDouble(amount)), interval:1h
```

```
// Average order value trend — daily for the past week
fetch bizevents, from:-7d
| filter event.type == "com.myapp.order.completed"
| filter isNotNull(amount)
| fieldsAdd day = bin(timestamp, 24h)
| summarize {aov = avg(toDouble(amount)),
           daily_revenue = sum(toDouble(amount)),
           orders = count()}, by:{day}
| sort day asc
```

4. Error Rate KPIs

Business process error rates measure the percentage of transactions that fail. This differs from HTTP error rates — a payment can fail (business error) even when the HTTP call succeeds (200 OK).

```
// Business error rate by event type
// Assumes failures are their own event type (com.myapp.payment.failed)
fetch bizevents, from:-24h
| filter in(event.type, {"com.myapp.payment.processed", "com.myapp.payment.failed"})
| summarize {total = count(),
           failures = countIf(event.type == "com.myapp.payment.failed")}
| fieldsAdd error_rate_pct = round(toDouble(failures) / toDouble(total) * 100, decimals: 2)
```

```
// Error rate trend over time — detect spikes
fetch bizevents, from:-24h
| filter in(event.type, {"com.myapp.payment.processed", "com.myapp.payment.failed"})
| makeTimeseries {total = count(),
                  failures = countIf(event.type == "com.myapp.payment.failed")},
                 interval:1h
```

```
// Error rate by provider — which channels have the most failures?
fetch bizevents, from:-24h
| filter in(event.type, {"com.myapp.payment.processed", "com.myapp.payment.failed"})
| summarize {total = count(),
           failures = countIf(event.type == "com.myapp.payment.failed")}, by:{event.provider}
| filter total &gt; 10
| fieldsAdd error_rate_pct = round(toDouble(failures) / toDouble(total) * 100, decimals: 1)
| sort error_rate_pct desc
```

5. Metric Extraction via OpenPipeline

While DQL queries compute KPIs on demand, **OpenPipeline metric extraction** creates persistent Dynatrace metrics from business events. This enables:

- Long-term retention (metrics have longer default retention than events)
- Alerting via Dynatrace Intelligence or custom metric events
- Dashboard widgets with instant load times
- SLO definitions based on business metrics

OpenPipeline Configuration

In **Settings > Process and contextualize > OpenPipeline > Business events**, open the pipeline that processes your business events, go to the **Metric extraction** stage, and add two processors:

Processor	Matching condition	Metric key	Value field	Dimensions
Counter metric	`event.type == "com.myapp.order.completed"`	`bizevents.orders.count`	— (increments by 1 per matching record)	`event.provider`, `product_category`
Value metric	`event.type == "com.myapp.order.completed"`	`bizevents.orders.revenue`	`amount`	`event.provider`, `product_category`

OpenPipeline matchers accept **double-quoted** strings only — `event.type == 'com.myapp.order.completed'` is rejected.

> **Metric key prefix.** The classic-pipeline extraction page requires keys *"starting with the bizevents. prefix"*; no OpenPipeline page states a prefix rule for business-event metrics. Using `bizevents.` here is the safe choice — it satisfies the classic rule and keeps business metrics easy to find with `metrics | filter startsWith(metric.key, "bizevents.")`.

Benefits of Metric Extraction

Approach	Latency	Retention	Alerting	Cost
DQL on bizevents	Seconds	Event retention (default 35 days)	Manual threshold	Scans event data
Extracted metrics	Sub-second	Metric retention (15 months included, extendable to 10 years)	Native Dynatrace Intelligence/SLO	Pre-aggregated

> **Classic-pipeline variant — the bridge to classic dashboards.** Alongside the OpenPipeline path shown here, Dynatrace documents business-event metric extraction via the classic pipeline (Settings > Business Observability > Metric extraction). Rules there emit metrics keyed with the `bizevents.` prefix (up to 50 dimensions) that are readable in Data Explorer, classic dashboards, and classic metric-event alerting — useful when report consumers still live in the classic (Gen2) experience. It is closing, though — per Business event metric extraction via classic pipeline (DT docs): *"For new accounts created from September 2026, classic pipeline is not available. Use OpenPipeline to process your data."* and *"Processing with the classic pipeline will be deactivated in a future release."* Existing classic rules stop producing metrics once that deactivation reaches your environment, so build new extraction on the OpenPipeline path; either way the output is an ordinary metric. See **BIZEV-07: Gen2 vs Gen3 — Business Events Without (or Before) the Full Move** for the full hybrid-adoption pattern.

```
// If you have extracted business metrics, query them with timeseries
// Replace with your actual metric key. On a counter metric use sum(): count() returns the
// number of reported data points, not the number of orders.
timeseries order_volume = sum(bizevents.orders.count, default: 0), from:-24h
```

6. Business Health Score

A composite health score combines multiple KPIs into a single indicator. This simplifies executive reporting and enables at-a-glance health checks.

Example Scoring Model

KPI	Weight	Green	Yellow	Red
Transaction volume (vs baseline)	30%	> 90% of baseline	70-90%	< 70%
Error rate	30%	< 1%	1-5%	> 5%
Average order value (vs baseline)	20%	> 95% of baseline	80-95%	< 80%
Conversion rate (vs baseline)	20%	> 90% of baseline	70-90%	< 70%

```
// Compute a simple business health dashboard — key metrics at a glance
fetch bizevents, from:-1h
| summarize {total_events = count(),
           distinct_types = countDistinct(event.type),
           distinct_providers = countDistinct(event.provider)}
| fieldsAdd time_window = "Last 1 hour",
           status = if(total_events &gt; 0, then: "ACTIVE", else: "NO DATA")
```

```
// Multi-KPI summary — compute all key metrics in one query
fetch bizevents, from:-24h
| summarize {total_transactions = count(),
           order_count = countIf(event.type == "com.myapp.order.completed"),
           payment_failures = countIf(event.type == "com.myapp.payment.failed"),
           total_payments = countIf(in(event.type, {"com.myapp.payment.processed",
"com.myapp.payment.failed"}))}
| fieldsAdd error_rate = if(total_payments &gt; 0,
               then: round(toDouble(payment_failures) / toDouble(total_payments) * 100, decimals:
2),
               else: 0.0)
| fieldsAdd health = if(error_rate &lt; 1.0, then: "HEALTHY",
                     else: if(error_rate &lt; 5.0, then: "DEGRADED", else: "CRITICAL"))
```

7. Trend Analysis

Trends reveal whether KPIs are improving or degrading over time. Use `makeTimeseries` for visual trends and `summarize` with `bin()` for tabular comparisons.

```
// Weekly trend — transaction volume per day for the past 4 weeks
fetch bizevents, from:-28d
| fieldsAdd day = bin(timestamp, 24h)
| summarize daily_volume = count(), by:{day}
| sort day asc
```

```
// Week-over-week comparison using day-of-week grouping
fetch bizevents, from:-14d
| fieldsAdd week = getWeekOfYear(timestamp, timezone: "America/New_York"),
           dow = getDayOfWeek(timestamp, timezone: "America/New_York")
| summarize volume = count(), by:{week, dow}
| sort week asc, dow asc
```

```
// Timeseries trend of business events by type — 7 days, 6h intervals
fetch bizevents, from:-7d
| makeTimeseries event_count = count(), by:{event.type}, interval:6h
```

8. Summary and Next Steps

In this notebook, you learned:

- **Defining business KPIs** from event data — volume, revenue, error rates, conversion
- **Transaction volume analysis** — Counting, trending, and comparing event volumes

- **Revenue metrics** — AOV, hourly revenue, daily revenue trends
- **Error rate calculation** — Business-level failure rates distinct from HTTP errors
- **Metric extraction** — Using OpenPipeline to create persistent metrics from events
- **Health scoring** — Combining multiple KPIs into composite health indicators
- **Trend analysis** — Week-over-week and day-over-day comparisons

Next Steps

- **BIZEV-06: Executive Reporting** — Package these KPIs into executive-ready reports
- Consider setting up OpenPipeline metric extraction for your most important KPIs

References

- Dynatrace Business Analytics
- Metric extraction stage in OpenPipeline (DT docs) — *"The Counter metric processor increments a counter by 1 for each matching record."*
- DQL matcher in OpenPipeline (DT docs)
- Metrics FAQ (DT docs) — *"Grail metrics come with 15 months of included retention at 1-minute granularity by default. This retention can be increased to up to 10 years."*
- DQL structuring commands (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.